

CS 311
Design & Analysis of Algorithms

Execution Time of Algorithms, Exercises

execution time is the same as time complexity

1) Algorithm Sum (A,n)

{

s = 0;]1

for(i=0; i<n; i++)]n+1

s=s+A[i];]n

return s;]1

}

1+n+1+n+1 = 3+2n ⇒ O(n)

n = 3

i=0, 1, 2, 3 → false

2) for(i=n; n>0; i--)]n+1

st.; n

n+1+n=2n+1 ⇒O(n)

n = 3

i=3, 2, 1, 0 false

Execution Time of Algorithms, Exercises

3) Algorithm Sum (A, B, n)

```
{
for(i=0; i<n; i++)
    for(j=0; j<n; j++)
        C[i][j] =A[I,j] + B[i,j];
}
```

4) for(i=0; i<n; i+=2)

st.;

Execution Time of Algorithms, Exercises

5) `for(i=0; i<n; i++)` n+1

`for(j=0; j<i; j++)` n(n+1)

=> goes to $O(n^2)$

`st. ;`

Execution Time of Algorithms, Exercises

6) $p = 0;$

$for(i=1; p<n; i++)$ (when $p>n$, stop)

$p = p + i;$

$P = \sum k$

$P \simeq k^2$

$\sqrt{k^2} > \sqrt{n}$

$k > \sqrt{n} \rightarrow O(\sqrt{n})$

$i=1 \quad P=0$

i	P	# execution
1	0+1	1
2	1+2	1
3	1+2+3	1
⋮	⋮	⋮
	1+2+3+4	1
	1+2+3+4+5	1
⋮	⋮	⋮
k	1+2+3+...+k	1

Execution Time of Algorithms, Exercises

*7) for(i= 1; i < n; i = i*2)*

st. ;

logn

so O(logn)

Execution Time of Algorithms, Exercises

8) for(i= n ; i >= 1 ; i = i/2)

st. ;

$\log n$

Execution Time of Algorithms, Exercises

9) *for(i= 0 ; i < n ; i ++)* Jn

st. ;

for(j = 0 ; j < n ; j++) Jn

st. ;

not nested so O(n)

Execution Time of Algorithms, Exercises

10) $p = 0;$

$for(i = 1 ; i < n ; i = i*2)$ log n times

$p{+}{+} ;$ p = log n

$for(j = 1 ; j < p ; j = j*2)$ log p = log log n

$st. ;$

$T(n) = \log n + \log\log n$

↑ dominating/ bigger

So $O(n) = \log n$

Execution Time of Algorithms, Exercises

11) was on a major

```
Algorithm Method_2 (n)
  if n <= 1 then
     return 1
  else
     return Method_2(n/2) + Method_2(n/2)
```

$2T(\frac{n}{2}) \Rightarrow O(n)$

was in an old final

Execution Time of Algorithms, Exercises

```
12) Function Mystery1(A, n):
sum = 0                          → c
for i from 1 to n:               → n
  j = 1
  while j < i * i:               → n^2
    k = 1
    while k < j:
      sum = sum + A[i] * A[k]
      k = k * 2                  log n
  j = j + 1
```

n × n^2 × log n [nested loops] = n^3 log n

```
13) Function Mystery2(A, n):
sum = 0                          → c
for i from 1 to n:               → n
  j = 1
  while j < i:                   → n
    sum = Mystery1(A,j)          n^3 log n
    j = j + 1
```

n × n × n^3 log n = n^5 log n

Execution Time of Algorithms, Exercises

- *for(i=0; i<n; i++)* ⇒ O(n)
- *for(i=0; i<n; i+=2)* ⇒ O(n)
- *for(i=n; i<1; i--)* ⇒ O(n)
- *for(i=1; i<n; i=i*2)* ⇒ O(log n)
- *for(i=n; i<1; i=i/2)* ⇒ O(log n)

Execution Time of Algorithms, Exercises

- *for(i=0; i<n; i++)* → *O(n)*
- *for(i=0; i<n; i+=2)* → *n/2 >> O(n)*
- *for(i=n; i>1; i--)* → *O(n)* If it was i<1 it will never execute since n is a number >1 so it'll be false and never execute
- *for(i=1; i<n; i=i*2)* → *O(log2n)*
- *for(i=n; i<1; i=i/2)* → *O(log2n)*